Building a Portfolio-Ready Analytics App

A step-by-step guide for a resume-worthy data analytics project

1. Why This Project Works on a Resume

Most "analytics portfolio projects" are just a Jupyter notebook with a bar chart. What separates a *hireable* project is that it demonstrates the full pipeline: data engineering → analysis → product thinking → deployment. This guide builds a **self-serve sales/product analytics app** — a domain that's universally understood by recruiters (SaaS metrics, e-commerce, or marketing analytics all work).

By the end, you'll have:

A live, deployed app with a public link
A GitHub repo with clean documentation
3–4 strong resume bullet points backed by real, defensible work
Talking points for interviews (this matters more than the app itself)

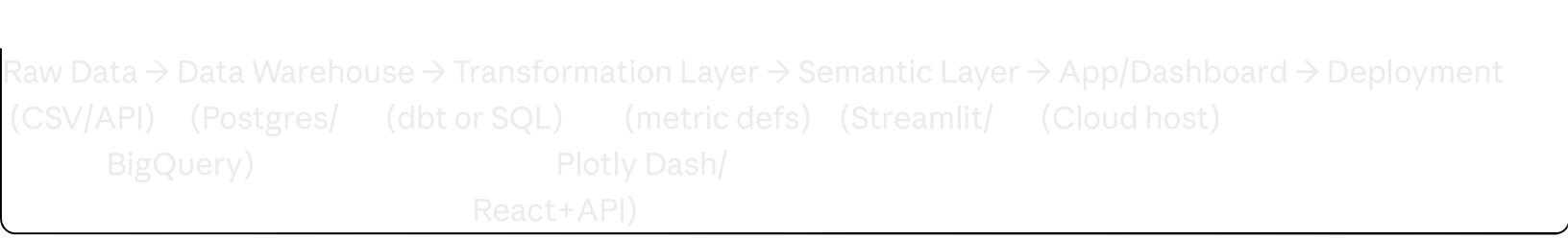
2. Choose Your Domain (Pick One)

Domain	Dataset Source	Good For
SaaS product analytics	Simulate or use Olist	PM/growth analytics roles
E-commerce sales	Olist, or Superstore dataset	Retail/BI analyst roles
Marketing funnel	Google Analytics sample dataset (BigQuery public)	Marketing analytics roles
Financial/ops metrics	Simulated transactions	Finance analyst roles

Recommendation: SaaS product analytics (MRR, churn, cohort retention, funnel conversion) is the most broadly impressive — it signals you understand business metrics, not just SQL syntax.

3. Architecture Overview





Recommended stack (balance of impressiveness vs. time cost):

Layer	Tool	Why
Storage	PostgreSQL (free tier on Supabase/Neon)	Real SQL, not just pandas
Transformation	dbt (or plain SQL scripts)	Shows modeling discipline, industry standard
Backend/logic	Python (pandas, SQLAlchemy)	Universal, recruiter-legible
App layer	Streamlit or Plotly Dash	Fast to build, genuinely interactive, easy to deploy
Deployment	Streamlit Community Cloud / Render / Vercel	Free, gives you a public link
Version control	GitHub with a real README	This *is* part of the deliverable

If you want to signal frontend/product chops instead, swap Streamlit for a **React + Recharts + FastAPI** stack — more impressive to a tech-forward employer, but 3–4x the build time.

4. App Structure

Two structures matter here: the **repo/folder layout** (what a hiring manager sees on GitHub) and the **UI/page structure** (what a user sees when they open the app). Get both right and the project reads as a real product, not a script.

4.1 Repository / Folder Structure



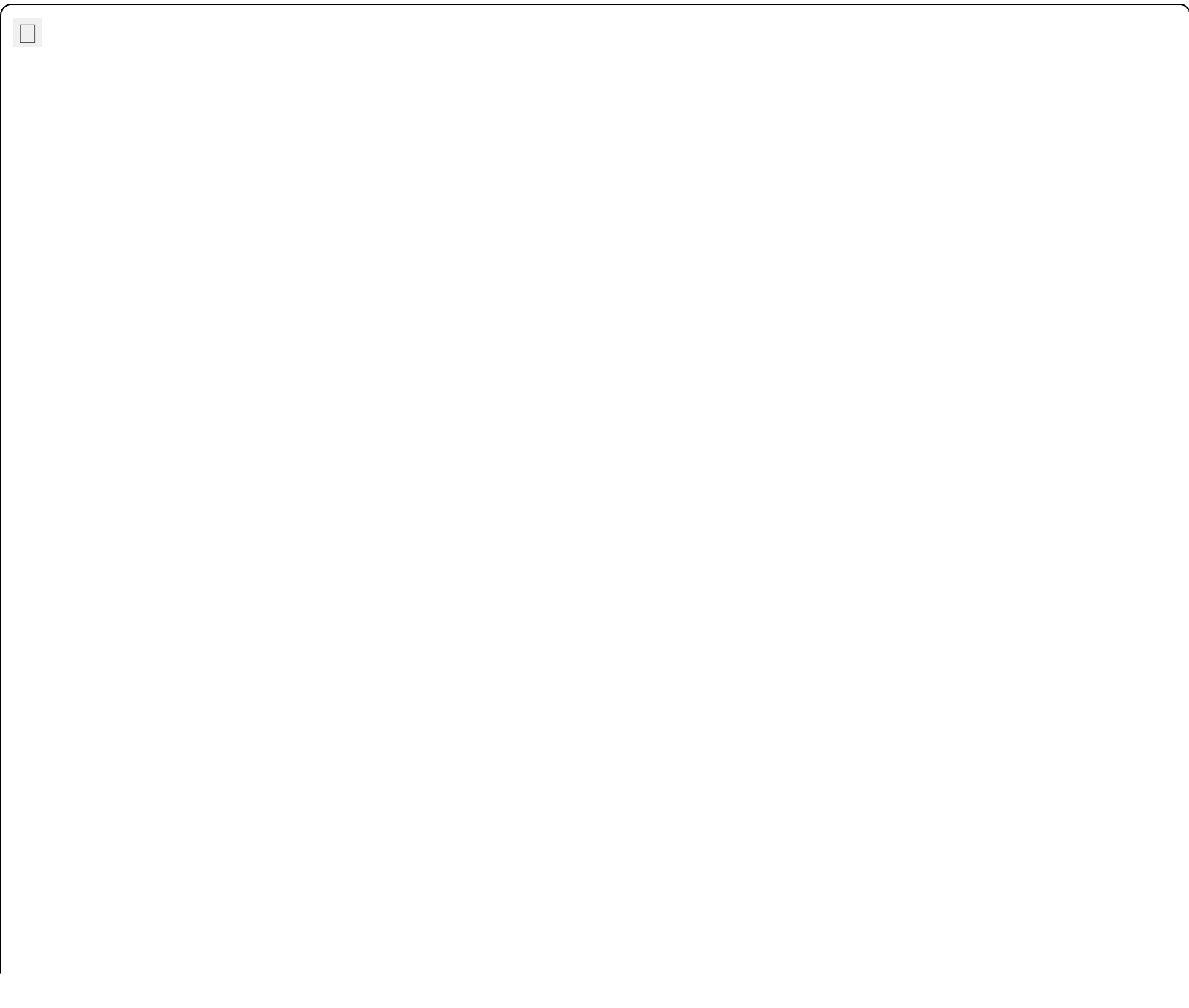
```
analytics-app/
├── README.md                  # Problem, architecture diagram, screenshots, demo link
├── data_dictionary.md         # Every table & column, defined
├── .env.example               # DB connection template (never commit real .env)
├── requirements.txt
│
├── data/
│   ├── raw/                   # Original source files (or scripts to pull them)
│   └── seed/                  # Synthetic data generator scripts, if used
│
├── db/
│   ├── schema.sql             # Raw table definitions
│   └── models/                # dbt models (or plain SQL views)
│       ├── staging/           # 1:1 cleaned versions of raw tables
│       │   ├── stg_users.sql
│       │   └── stg_events.sql
│       └── marts/             # Business-logic layer — your semantic layer
│           ├── metrics_mrr.sql
│           ├── metrics_churn.sql
│           ├── metrics_retention_cohorts.sql
│           └── metrics_funnel.sql
│
├── src/
│   ├── db.py                  # Connection/session handling
│   ├── queries/               # Parameterized query functions, one per metric
│   │   ├── trends.py
│   │   ├── cohorts.py
│   │   ├── funnel.py
│   │   └── anomalies.py
│   └── utils/
│       └── formatting.py      # Number/date formatting helpers shared across pages
│
├── app/
│   ├── Home.py                # Streamlit entrypoint — headline metrics view
│   └── pages/
│       ├── 1_Retention.py     # Cohort heatmap
│       ├── 2_Funnel.py        # Funnel + drop-off view
│       ├── 3_Segments.py      # Breakdown by plan/channel/geography
│       └── 4_Alerts.py        # Anomaly/alerts panel
│
├── tests/
```

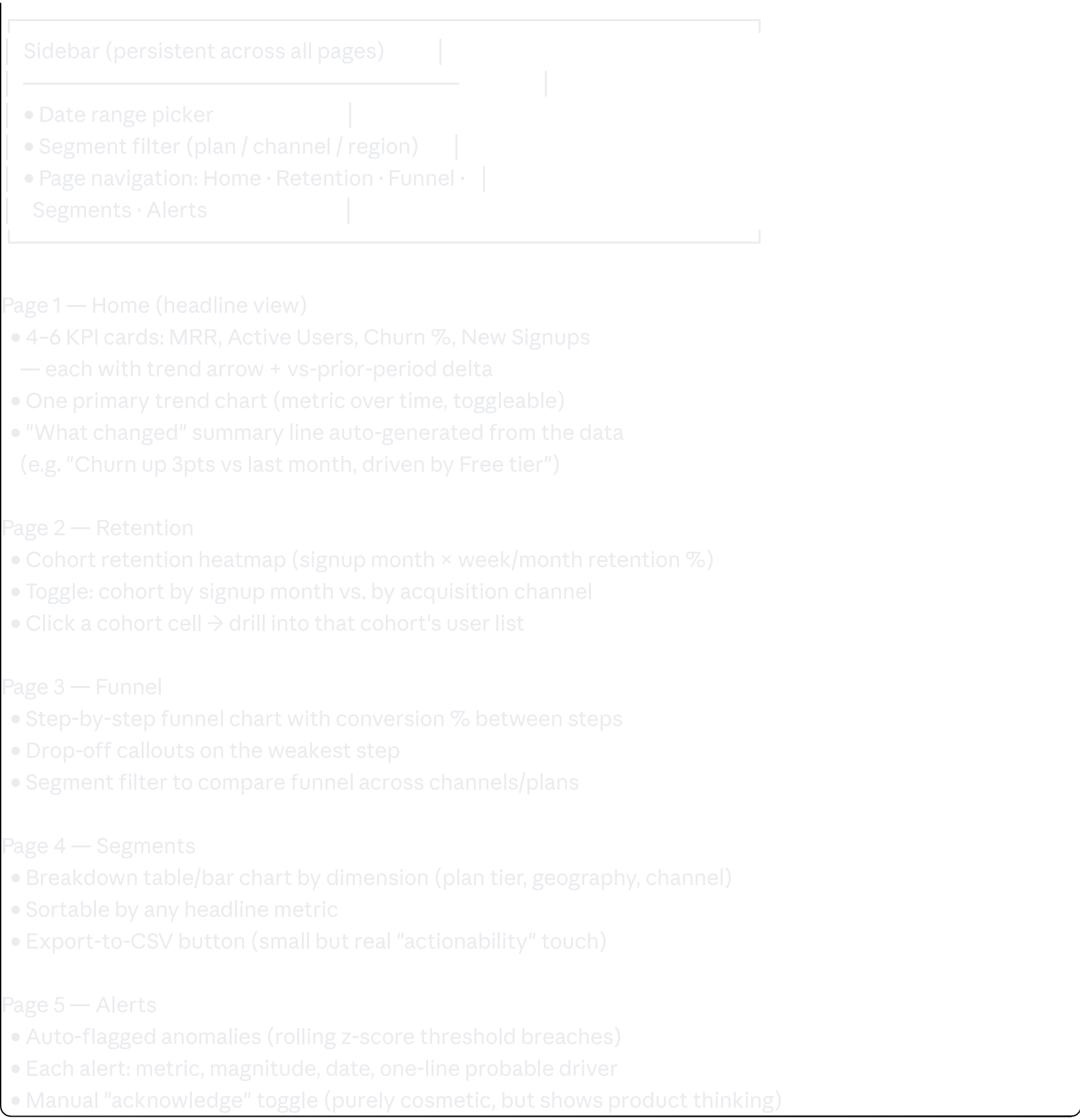
```
│   ├── test_metrics.py          # Unit tests on metric calculations
│   └── test_data_quality.py     # Null/duplicate/referential checks
│
└── .github/
    └── workflows/
        └── ci.yml               # Runs tests on every push (stretch goal)
```

Why this layout reads well to reviewers: `db/models/staging` vs `marts` mirrors real dbt convention; `src/queries` separates data logic from UI code (a junior mistake is jamming SQL directly into the UI file); `tests/` signals you think about correctness, not just output.

4.2 App / UI Structure (What the User Sees)

Structure the app as a small set of focused screens, not one cluttered page — this mirrors the "speed to insight" principle from the design guide.





Design rule of thumb: every page answers one question. Home = "are we healthy right now?" Retention = "are we keeping people?" Funnel = "where do we lose people?" Segments = "who specifically?" Alerts = "what needs my attention today?" If a chart doesn't clearly serve one of those questions, cut it.

5. Step-by-Step Build Plan

Phase 1 — Data Foundation (Days 1–2)

Pick and download your dataset (or generate synthetic data with `faker`/`numpy` if you want cleaner control over the story).
Load raw data into PostgreSQL. Don't skip this step and just use CSVs in pandas — "worked with a database" is a real resume line.
Write a data dictionary (`data_dictionary.md`) documenting every table and column. This shows rigor.

Phase 2 — Define Your Metrics Layer (Days 2–3)

This is the step most portfolio projects skip, and it's the one that signals seniority.

Write explicit definitions before writing any code, e.g.:

Active user = logged in ≥1x in trailing 28 days
Churn = no activity in 30 days after being active
MRR = sum of active subscription value, normalized to monthly
Conversion rate = signups who complete activation event / total signups

Store these as SQL views or dbt models (`models/marts/metrics.sql`) — this becomes your "semantic layer," a legitimate industry term you can use in interviews.

Phase 3 — Core Analysis (Days 3–5)

Build the underlying SQL/Python that powers each feature:

Trend analysis — metric over time, with period-over-period comparison
Cohort retention — a cohort grid (signup month × retention %) — this is the single most impressive chart you can put on a data analytics resume
Funnel analysis — step-by-step conversion with drop-off %
Segmentation — breakdown by plan tier, geography, acquisition channel
Anomaly flagging — simple z-score or rolling-average threshold to flag unusual days

Phase 4 — Build the App (Days 5–8)

Structure the app around the principles from the earlier answer:

Home view: 4–6 headline metrics with trend arrows and vs.-prior-period deltas (not 20 charts)
Drill-down view: filters for date range, segment, channel
Retention view: cohort heatmap
Funnel view: interactive funnel chart
Alerts panel: flags anomalies automatically, e.g. "Signups dropped 18% vs. last week — investigate"

Keep interactions fast — pre-aggregate where possible rather than querying raw rows live.

Phase 5 — Polish & Deploy (Days 8–9)

Add filters, tooltips, and empty/loading states — this is what makes it look like a product, not a script.
Write a clear `README.md`: problem statement, architecture diagram, screenshots, how to run locally, live demo link.
Deploy (Streamlit Cloud is the fastest path — free, GitHub-connected, live in minutes).
Record a 60–90 second Loom/video walkthrough. Link it in your README and resume — many recruiters won't click into a live app, but will watch a video.

6. What to Put on Your Resume

Use outcome-oriented, specific language — not "built a dashboard."

Example bullets:

Designed and deployed a self-serve product analytics application (PostgreSQL, dbt, Streamlit) modeling MRR, churn, and cohort retention across 50K simulated user records, with a defined semantic layer ensuring consistent metric definitions.

Built cohort retention and funnel-conversion analysis pipeline in SQL/Python, surfacing a 22% week-1 drop-off point and translating it into an actionable segmentation view.

Implemented automated anomaly detection (rolling z-score) to flag metric deviations, reducing manual dashboard-checking to alert-driven review.

Each bullet should map to something you can talk through in an interview for 3–5 minutes.

7. Interview Talking Points to Prepare

Interviewers will probe for judgment, not just execution. Be ready to answer:

Why did you choose these specific metrics?
How did you decide on the retention/churn definition? What are the edge cases?
What would you do differently with real (not synthetic) data at 10x scale?
How would this app handle 10M rows instead of 50K — what breaks first?
What's one insight the app surfaced that you didn't expect?

That last one matters most — have a real answer. Even in synthetic data, find *something* — a channel that converts worse, a cohort that churns faster — and be able to explain it.

8. Time & Difficulty Estimate

Skill Level	Timeline
Beginner (comfortable with pandas/SQL basics)	3–4 weeks, part-time
Intermediate	1.5–2 weeks
Advanced (add dbt, tests, CI/CD, React frontend)	3+ weeks, but strongest resume signal

Scope it down before scoping it up: a working end-to-end app with 4 clean metrics beats an ambitious half-finished app with 15.

9. Stretch Goals (If You Want to Stand Out Further)

Add **dbt tests** (uniqueness, not-null, referential integrity) — signals data engineering maturity
Add a **CI pipeline** (GitHub Actions) that runs tests on push
Add **user authentication** so it behaves like a real multi-tenant product
Write a **one-page "PRD"** for the app as if you were the PM — shows product thinking, not just execution